

A pipeline-rewrite operator

Document #: P2011R1
Date: 2020-04-16
Project: Programming Language C++
Audience: EWG
Reply-to: Colby Pike
<vectorofbool@gmail.com>
Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Revision History](#)
- [2 Abstract](#)
- [3 Introduction and Motivation](#)
 - [3.1 Pipeline Style](#)
 - [3.2 Supporting `|` as an Pipeline Operator](#)
 - [3.3 Problems with `|` as a Pipeline Operator](#)
 - [3.4 Ambiguity problems with `|` as a Pipeline Operator](#)
- [4 Proposal: Rewriting Pipelines with a `|>` Operator](#)
 - [4.1 A few more examples](#)
 - [4.2 Further examples](#)
 - [4.3 Prior Art \(in C++ and elsewhere\)](#)
- [5 Operator Precedence](#)
- [6 A Placeholder syntax](#)
- [7 Concerns with the Pipeline Operator](#)
 - [7.1 What about Unified Function Call Syntax?](#)
 - [7.2 What about pipeline composition?](#)
 - [7.3 What to do about Ranges going forward?](#)
 - [7.4 Other Concerns](#)
- [8 Wording](#)
- [9 Acknowledgments](#)
- [10 References](#)

§ 1 Revision History

R0 of this paper [[P2011R0](#)] was presented in Prague [[prague.minutes](#)]. The room encouraged further work on the proposal (16-0) and the discussion largely focused on the question of operator precedence, where we were asked to explore giving `|>` a lower precedence than `.` or `->` (18-0). That question was also raised on the reflectors [[ext.precedence](#)].

This revision lowers the precedence of `|>` (as described in [operator precedence](#) and includes discussions of [what to do about Ranges pipelines](#) in C++23 and also considers the idea of using a [placeholder syntax](#).

This revision also expands the kinds of expressions that can be pipelined into to additionally include casts and explicit type conversion.

§ 2 Abstract

This paper proposes a new non-overloadable operator `|>` such that the expression `x |> f(y)` evaluates exactly as `f(x, y)`. There would be no intermediate subexpression `f(y)`.

This is notably unrelated to, and quite different from, [\[P1282R0\]](#), which proposed an overloadable `operator|>`, allowing the evaluation of the above expression as `operator|>(x, f(y))`.

§ 3 Introduction and Motivation

While the addition of Ranges into the standard brings many great features and concepts, the “pipeline” features could use some attention. The current incarnation of the pipeline functionality brings a few important drawbacks:

1. The necessary amount of supporting code machinery lends itself to high amounts of complexity, creating a larger maintenance burden on developers who wish to use and write pipeline-style code.
2. The support machinery can incur large amounts of overhead when inlining and peephole optimizations are not enabled.
3. The support machinery places additional large burdens on the implementation in that it needs to parse and process large amounts of the support code that is needed to support the syntax.
4. Adopting the ability to add pipeline support for some algorithms might be impossible given [fundamental ambiguities](#).

The goal of the “pipeline-rewrite” operator proposed herein is to solve all of the above issues, as well as generalize the concept of “pipeline” code to work with arbitrary functions and types, and not just those that must specifically request it. We elaborate on some of these issues when we talk about the [problems with |> as a pipeline operator](#).

The addition of a “pipeline-rewrite” operator requires no API adjustments to any existing or proposed libraries in order to support such an operator.

§ 3.1 Pipeline Style

We have seen the proliferations of generic algorithms being implemented as free functions. For example, where many languages have a single type to represent a “sequence” of values, C++ permits an unlimited number of “sequence” types tailored to the needs of their respective domain, and the generic algorithms that operate on them work identically (provided the underlying type meets the appropriate guarantees). In classical object-oriented languages, the algorithms are attached to the objects themselves. For example:

```
// Some JavaScript
const seq = [1, 2, 3, 4];
const twice = seq.map(i => i * 2);
```

Here `map` is a *member* of `seq`, despite the concept of “mapping” being entirely abstract.

In many languages, when new sequence types are needed, they may be defined, but can suffer from performance penalties, but even worse: The algorithms are gone! The algorithm methods need to be re-implemented again on the new types.

The C++ standard library instead opts for generic free functions. These have great benefits, including supporting containers of disparate types:

```
QList<int> integers = get_integers();
std::vector<int> twice;
std::transform(begin(integers), end(integers), back_inserter(twice),
    [](int i){ return i*2; });
```

Much of the standard library accepts “iterator pairs” as their representation of a sequence. This has some benefits, such as the algorithms not needing to know anything about the underlying container. This also has some drawbacks, such as algorithms not being able to know anything about the underlying container.

One of the biggest drawbacks, though, is the simple verbosity. We do not often write application code dealing strictly with iterator pairs. Instead, we’ll be using actual concrete data structures that expose the iterator pairs that we hand to algorithms.

Amongst many other things, `Ranges` defines new overloads for many of the standard algorithms that accept range types which represent iterator pairs (or an iterator and a sentinel, but that isn’t relevant).

```
QList<int> integers = get_integers();
std::vector<int> twice;
ranges::transform(integers, back_inserter(twice),
    [](int i){ return i*2; });
```

Another idea introduced by `Ranges` is the composition of algorithms.

Here, we will borrow one of the most famous examples from the `ranges-v3` library: The calendar printing example [[range.calendar](#)]. We will start with a very uglified version of the example’s apex, `format_calendar`:

```
template <typename Calendar>
auto format_calendar(size_t months_per_line, Calendar&& cal) {
    // Group the dates by month
    auto months = by_month(cal);
    // Format the months into a range of strings
    auto month_strings = layout_months(months);
    // Group the months that belong side-by-side
    auto chunked_months = chunk(month_strings, months_per_line);
    // Transpose the rows and columns of side-by-side months
    auto transposed = transpose_months(chunked_months);
    // Ungroup the side-by-side months
    auto joined_view = view::join(transposed);
    // Join the strings of the transposed months
```

```

    return join_months(joined_view);
}

```

This code is not inscrutable, but it is far from what the original looked like. We have a several variables that are essentially meaningless, as their names are tautological to the spelling of their initializing expression. And because these variables are only used in the immediately following line, we may as well place each variable’s initializer in place of the variable name in the following call. The result is horrific, to say the least:

```

template <typename Calendar>
auto format_calendar(size_t months_per_line, Calendar&& cal) {
    // Join the strings of the transposed months
    return join_months(
        // Ungroup the side-by-side months
        view::join(
            // Transpose the rows and columns of side-by-side months
            transpose_months(
                // Group the months that belong side-by-side
                chunk(
                    // Format the months into a range of strings
                    layout_months(
                        // Group the dates by month
                        by_month(cal)
                    ),
                    months_per_line
                )
            )
        );
};
}

```

(Our favorite feature of the above horror is the `months_per_line` appearing quite distant from the function call to which it is an argument.)

While the code is frightening, it is conceptually equivalent to the prior example. Both of these examples are very dissimilar to the code found in the range-v3 [\[range-v3\]](#) example upon which they were based.

Ranges also seeks to tackle the above problem with the idea of pipeable objects.

Pipeline-style is an increasingly popular way to write code, especially in functional programming languages. Ranges provides pipeline style via overloading of the bitwise-or `|` binary operator. In the pipeline style, the value on the left of the “pipeline” operator is conceptually “fed into” the expression on the right, where the right-hand-side is some “partial” expression missing the primary argument on which it operates. The actual example from range-v3 uses this syntax to produce the much more concise and readable pipeline style:

```

auto
format_calendar(std::size_t months_per_line)
{
    return make_pipeable([=](auto &&rng) {
        using Rng = decltype(rng);
        return std::forward<Rng>(rng)
            // Group the dates by month:
    });
}

```

```

        | by_month()
        // Format the month into a range of strings:
        | layout_months()
        // Group the months that belong side-by-side:
        | chunk(months_per_line)
        // Transpose the rows and columns of the size-by-side months:
        | transpose_months()
        // Ungroup the side-by-side months:
        | view::join
        // Join the strings of the transposed months:
        | join_months();
    });
}

```

Usage of `format_calendar` also makes use of the “pipeline” syntax:

```

copy(dates(start_date, end_date) | format_calendar(3),
    calendar_lines);

```

Where `dates` lazily generates date objects which are fed into the `format_calendar` algorithm.

Although the above examples use ranges, the pipeline style can be applied to any type of objects, from integers to strings to database rows.

§ 3.2 Supporting `|` as an Pipeline Operator

How does `|` work in the above examples, and with Ranges in general? After all, it’s just the bitwise-or operator. The “pipeline” semantics aren’t built into the language.

The answer, of course, is to use operator overloading. To support `transform(rng, projection)` and `rng | transform(projection)`, the `transform` name does not correspond to a single function. It must instead name an overload set (or, as with everything in Ranges, a single object with multiple `operator()` overloads). The type returned by the two overloads is radically different. The partially-applied form intended for use with `|` stores its argument in an object which defines the overloaded `operator|`. If a range is given as the left-hand operand of the `|` operator, only then is the algorithm fully-applied and ready to produce results.

Let’s go through what the implementation of this machinery looks like so that we can point out what the many issues with it are.

For the full function call, `transform(rng, projection)`, we can provide complete constraints. We have all the information we need: we know the range, we know the function type, and we know that the function type has to be compatible with the range. With concepts, we just need to write those constraints out:

```

template <std::ranges::viewable_range R,
          std::regular_invocable<std::range_reference_t<R>> F>
auto transform(R&& range, F&& projection)
    -> transform_view<std::ranges::all_view<R>, std::decay_t<F>>
{
    // implementation
}

```

Now the implementation of `transform_view` itself is non-trivial - but its implementation is unrelated to the topic we're focusing on here.

Now for the partial function call, in order to support `rng | transform(f)` we need to first support `transform(f)`. This call does not have complete information - we just have a function, but we cannot check this function all by itself. We can only check if a type is callable with a specific argument, we cannot check in general if a type is callable at all. There is no constraint that we can add on this type at all (outside of it being copyable), so we're left to just write:

```
#define FWD(x) static_cast<decltype(x)&&>(x)

template <typename F>
auto transform(F&& projection)
{
    return make_left_pipeable(
        [f=std::forward<F>(projection)](std::ranges::viewable_range auto&& r)
        -> decltype(transform(FWD(r), std::ref(f)))
        {
            return transform(FWD(r), std::ref(f));
        });
}
```

This is probably the most concise way to implement partial transform: we have a utility `make_left_pipeable` which takes a lambda and returns a type that, when left-`|`-ed invokes the lambda. This lambda has to be SFINAE-friendly, and needs to just forward all of its arguments properly to the complete call so as to avoid having to duplicate the constraints.

§ 3.3 Problems with `|` as a Pipeline Operator

There are a few facially obvious drawbacks to using `|` for pipeline semantics:

- The code required to support using `|` functionality adds overhead during compilation, and without the aide of the inliner and basic optimizations it can be expensive at runtime.
- Defining new range algorithms necessitates opting-in to this machinery. Existing code cannot make use of pipeline style.
- Supporting both pipeline style and immediate style requires algorithms to provide both partial and full algorithm implementations, where the partial implementation is mostly boilerplate to generate the partially applied closure object.

We showed in the previous section how a `|`-friendly `transform_view` could be implemented. Could this be improved in a different way, perhaps with reflection? This seems, at least, theoretically possible. The algorithm would be: take the “full call”, strip the first argument and strip all the constraints that (recursively) require the first argument). Then have the body be a lambda that takes a single argument, captures the function parameters by forward, and re-instances the constraints that were stripped from the original algorithm. A much simpler (and probably *mostly*) would be to generate a partial call version that is:

```
auto transform(auto&&... args)
{
```

```

return make_left_pipeable(
    [...args=FWD(args)](/* no constraint */ auto&& first)
    -> decltype(transform(FWD(first), FWD(args)...))
    {
        return transform(FWD(first), FWD(args)...);
    });

```

This one could actually be written as a macro once we write `make_left_pipeable` one time.

But even with the simple macro approach (and, spoiler alert, that direction as a whole may not be viable), there's still cost to consider:

- while `views::transform(rng, f)` and `rng | views::transform(f)` generate identical code in optimized builds, the latter has overhead in debug builds - both in terms of performance and debugability. That makes it not entirely the zero-overhead abstraction that we like to claim, for those users that regularly use debug builds (which is a large percentage of users).
- having to deal with the extra `|` abstraction requires extra parsing on the compiler's part for each TU (possibly less of an issue in a modules world)
- even with modules, this abstraction has compile time cost as `transform` now has two overloads (instead of one) regardless of if we use the pipeline syntax or not - and then we need to do an additional overload resolution for the `|`, all of which are function templates that need to be instantiated.

We'll discuss the second and third points more in the context of what we propose to do about [Ranges going forward](#).

Those are, in of themselves, far from ideal. But these problems are all just work - work for the programmer to write the initial `|` support to begin with (just the one time), work for the programmer to write each additional overload to opt-in, work for the compiler to compile all these things together, work for the optimizer to get rid of all these things, work for the programmer to try to figure out what's getting called where.

§ 3.4 Ambiguity problems with `|` as a Pipeline Operator

But there's a bigger problem here that no amount increased programmer or compiler throughput can solve: sometimes the partial and complete calls are ambiguous.

Specific to ranges, the "total call" syntax always takes a range followed by some amount of helper arguments. In the `transform` case, this is a range followed by an invocable. The "partial call" syntax drops the range, and takes the rest of the arguments. What happens, though, when the second argument can itself be a range? How do you know? There are several examples where this comes up:

- `zip` is a view that takes a bunch of ranges and turns them into a range of tuples of references, it's an extremely useful range adaptor - just one that we won't quite have in C++20. `zip(a)` is itself a valid (if odd) view: just takes all the elements of a range and wraps them in a tuple of a single element. As a result, you cannot make both `zip(a, b)` and `a | zip(b)` yield a range of tuples, you have to pick one. `range-v3`'s `zip` does not support the pipeline syntax.

- `concat` is a view that takes a bunch of ranges and concatenates them all together in a larger range. Much like `zip`, `concat(a)` is a valid expression and so `a | concat(b)` can't work. `range-v3`'s `concat` does not support the pipeline syntax.
- `transform` actually has two forms: a unary form and a binary form. C++20 adds an algorithm that is `ranges::transform(rng1, rng2, result, binary_op)`. But `views::transform(r, f)` could be a valid expression, so there may be an ambiguity with `a | views::transform(b, f)`, if `f` happens to be invocable both as a unary and as a binary function. `range-v3` does not have a binary `transform` adapter, only a unary one.
- `views::join` in C++20 takes a range of ranges and collapses it to a single range, taking no additional arguments. One potentially useful argument it could take is a delimiter: `join(rng_of_rngs, delim)` could interleave the delimiter in between each element. But can itself be a range of ranges, then you'll run into problems with `rng | join(delim)`. `range-v3` only considers the pipeable form for non-joinable-ranges. An alternative approach might have been to just name them differently - `join(rng)` is unambiguous with `rng | join` and `join_with(rng, delim)` is unambiguous with `rng | join_with(delim)`.

Each of these are specific cases that have to be considered by the library author, which is just a lot of extra mental overhead to have to deal with. But it's somehow even worse than that.

In `range-v3` and in C++20, the only algorithms that opt-in to the pipeline syntax are those algorithms that take one or more ranges as input and produce a [lazy] range as output. There is a whole other class of algorithms that does not have this particular shape, but still would be quite useful to have pipeline syntax for. Conor Hoekstra, in his CppNow talk entitled Algorithm Intuition [[hoekstra](#)], presented a problem which boiled down to checking if a string had at least 7 consecutive 0s or 1s. One way to solve that using the pipeline syntax would be:

```
auto dangerous_teams(std::string const& s) -> bool {
    return s
        | views::group_by(std::equal_to{})
        | views::transform(ranges::distance)
        | ranges::any_of([](std::size_t s){
            return s >= 7;
        });
}
```

At least, it would be a way of solving this problem if the `any_of` algorithm had opted-in to the pipeline syntax. It does not. And doing so takes work - we have to go through all of the over 100 algorithms in the standard library and add these extra partial overloads, specify how all of them work, and then figure out how to deal with ambiguities. This is a lot of work for a large number of people, and still wouldn't help solve the problem for any of the algorithms in user code. And you can still get an ambiguity:

- `accumulate` does not currently have a range-based overload in C++20, but it needs to take an initial value as the second argument. If that initial argument is a range that happens to be addable, that's ambiguous. One such type is `std::string`: is `accumulate(str)` a complete call summing that string, or is it a partial call simply providing the initial value to the accumulator. You could work around this - either by not providing a default value for the initial value, or not providing a default value for the binary operator (or both). But either way, you need *some* workaround. `range-v3` does not have a

pipeable `accumulate`, but it only defaults the binary op and not the initial value - which itself prevents you from writing `accumulate(some_ints)`.

Rather than committing to:

- a lot of committee time to discuss opting in algorithms to the pipeline syntax, and
- a lot of committee time to discuss the right way to handle ambiguities, and
- a lot of library implementer time to implement the ones that we adopt, and
- a lot of compiler time compiling code that uses the pipeline syntax, and
- a lot of non-committee time to do the same for their own algorithms, including having to come up with mechanisms to support the pipeline syntax for algorithms like `zip` and `concat` too.

We would like to propose something better.

§ 4 Proposal: Rewriting Pipelines with a `|>` Operator

We propose a new, non-overloaded function call operator spelled `|>`. What it does is simply evaluate:

```
x |> f(y)
```

as

```
f(x, y)
```

Without any intermediate operands. That is, it rewrites code written in a pipeline style into immediate function call style. This rewriting of pipeline-style is why the name “pipeline-rewrite” was chosen.

It’s important to be clear that the above **does not** evaluate as:

```
operator|>(x, f(y))
```

There is no `f(y)` operand, there is no lookup for `operator|>` (this paper is proposing that such a declaration be ill-formed - this operator is not overloadable).

In other words, the following program is valid:

```
constexpr int add(int a, int b) {  
    return a + b;  
}  
  
constexpr int sum = 1 |> add(2);  
static_assert(sum == 3);
```

This is a complete program, no `#includes` or `imports`, no additional library glue necessary to make this work. The assertion directly invokes `add(1, 2)` just as if the code had been written that way to begin with. Indeed, an attempt at invoking a unary `add(2)` would be ill-formed!

This is similar to member function call syntax, where `c.f(x)` does not evaluate as the expression `operator.(c, f(x))` and instead evaluates as something much closer to `C::f(c, x)`

§ 4.1 A few more examples

The description above is roughly the entirety of the proposal. We take the expression on the left-hand side of `|>` and treat it as the first argument of the call expression on the right-hand side of `|>`.

Some more examples:

```
namespace N {
    struct X { int i; };
    int f(X x) { return x.i; }
}

template <typename T>
void f(T);

N::X{1} |> f(); // ADL finds N::f, is 1
N::X{1} |> (f)(); // parens inhibit ADL, so this calls `::f`, is a void

// immediately invokes the lambda with arguments 1 and 2
1 |> [](int i, int j){ return i + j; }(2);

// immediately invokes the lambda with the argument 1
1 |> [](int i){ return i; }();

template <typename T>
int g(T);

2 |> g<int>(); // equivalent to g<int>(2)

// arbitrary expressions can be composed in parens
template <typename F, typename G>
auto operator>>(F f, G g) {
    return [=](auto arg){
        return g(f(arg));
    };
}

// evaluates as dbl(add1(4))
auto add1 = [](int i){ return i+1; };
auto dbl = [](int i) { return i*2; };
4 |> (add1 >> dbl)();
```

All of the above works directly out of the box with this proposal.

§ 4.2 Further examples

§ 4.2.1 Inside out vs left-to-right

Consider trying to trim a `std::string`. We could write it this way:

```

auto trim(std::string const& str) -> std::string
{
    auto b = ranges::find_if(str, isalpha);
    auto e = ranges::find_if(str | views::reverse, isalpha).base();
    return std::string(b, e);
}

```

It's hard to interpret what's going on in that second line due to the inside-out reading that is necessary - there's a lot of back and forth. With the pipeline rewrite operator, we could rewrite this function to be entirely left-to-right:

```

auto trim(std::string const& str) -> std::string
{
    auto b = str |> ranges::find_if(isalpha);
    auto e = str |> views::reverse() |> ranges::find_if(isalpha);
    return std::string(b, e.base());
}

```

This ordering is a more direct translation of the original thought process: we take our string, reverse it, find the first alpha character, then get the base iterator out of it.

To make the `ranges::find_if` algorithm work with the `|>` operator, we need to write this additional code:

```

// (This space intentionally left blank)

```

That's right! Nothing at all!

Remember that the semantics of `|>` will *rewrite* the code:

```

// This:
auto e = str |> views::reverse() |> ranges::find_if(isalpha);
// becomes this:
auto e = ranges::find_if(views::reverse(str), isalpha);

```

That is, using `|>` is equivalent to the code not using the pipeline style.

§ 4.2.2 Using copy

Let's look at a non-lazy copy function:

```

template <typename Range, typename Output>
auto copy(Range&& rng, Output out) {
    for (const auto& item : std::forward<Range>(rng)) {
        *out++ = item;
    }
    return out;
}

```

This function operates on a range as its first argument, and an output iterator as its second argument. Usage is very simple:

```

std::vector<int> copies;
auto integers = get_integers();
copy(integers, back_inserter(copies));

```

We can elide the extraneous integers variable to shrink the code:

```
std::vector<int> copies;
copy(get_integers(), back_inserter(copies));
```

We may want to use pipeline syntax to perform the copy. Instead of using `|` for the pipeline style, we just use `|>`. That would look like this:

```
std::vector<int> copies;
get_integers() |> copy(back_inserter(copies));
```

§ 4.2.3 transform

One of the most fundamental algorithms is transform. It applies a projection function to each element of the input range and yields the result of that projection.

```
template <typename Range, typename Proj>
struct __transform_view {
    // ...
};

template <typename Range, typename Proj, typename Out>
auto transform(Range&& rng, Proj&& fn) {
    return __transform_view(rng, fn);
}
```

This algorithm is a *lazy* version of transform. It will apply the projection function to elements of rng as iterators on the __transform_view object are advanced.

Range algorithms compose. We can use this with copy to make a meaningful program:

```
copy(transform(get_words(), make_uppercase), ostream_iterator<string>{cout, "\n"});
```

This code, of course, is inside-out from how evaluation is ordered. We can feed the result of transform into copy using `|>`:

```
transform(get_words(), make_uppercase)
    |> copy(ostream_iterator<string>{cout, "\n"});
```

And, without writing any additional support code, we can use `|>` to feed get_words into transform:

```
get_words()
    |> transform(make_uppercase)
    |> copy(ostream_iterator<string>{cout, "\n"});
```

§ 4.2.4 A New Algorithm: each_as

Ranges will be receiving a function template to that creates a concrete range from another range. A very primitive implementation of one overload might look like this:

```
template <typename Container, typename Range>
Container to(const Range& rng) {
    Container ret(rng.begin(), rng.end());
```

```

    return ret;
}

```

This simply takes a range and uses it to fill a container with the iterator-pair constructor present on many container types. Usage looks like this:

```

auto filenames = get_strings()
    |> to<vector<filesystem::path>>()
    |> transform(get_filename)
    |> to<vector<string>>();

```

However: The `to` algorithm, unlike `transform`, is *eager*. It consumes each element of the input immediately. This requires a concrete new container type that will eagerly allocate a buffer to hold the new objects. In the above snippet, all we are doing is obtaining the filenames of each file, and we do not actually care about the intermediate `std::vector`.

Note: The above example is illustrative. There are other ways to perform the necessary transform.

What we may want is a new lazy algorithm that simply converts each range element to a new type as they pass through. How could we define such an algorithm?

```

template <typename T, typename Range>
auto each_as(Range&& rng) {
    return rng |> transform([](const auto& item) { return T(item); });
}

```

With `|>` at our disposal, there is no need to offer two overloads of `each_as` for the two styles. The above overload happily works with `|>` pipeline style:

```

auto filenames = get_strings()
    |> each_as<filesystem::path>()
    |> transform(get_filename)
    |> to<vector<string>>();

```

Or non-pipeline style:

```

auto filenames =
    each_as<filesystem::path>(get_strings())
    |> transform(get_filename)
    |> to<vector<string>>();

```

§ 4.2.5 A New Algorithm: `copy_insert/copy_extend`

A common operation is to collect the results of multiple computations into a single container. We can define two new algorithms:

```

template <typename Range, typename Container, typename Iter>
void copy_insert(Range&& rng, Container& c, Iter it) {
    rng |> copy(inserter(c, it));
}

template <typename Range, typename Container>
void copy_extend(Range&& rng, Container& c) {

```

```

    rng |> copy_insert(c, c.end());
}

```

Again, we have `|>` syntax using normal functions and no special return types or expression templates.

Using them is very simple:

```

// We may use pipeline style:
void collect_filenames(filesystem::path dirpath, vector<string>& fnames) {
    filesystem::directory_iterator{dirpath}
        |> copy_extend(fnames);
}

// Or we may use classical style:
void collect_filenames(filesystem::path dirpath, vector<string>& fnames) {
    copy_extend(
        filesystem::directory_iterator{dirpath},
        fnames
    );
}

```

§ 4.2.6 Not A New Algorithm: any_of

Of course, we can go back to Conor's example and provide a complete implementation of it:

```

auto dangerous_teams(std::string const& s) -> bool {
    return s
        |> views::group_by(std::equal_to{ })
        |> views::transform(ranges::distance)
        |> ranges::any_of([](std::size_t s){
            return s >= 7;
        });
}

```

It is worth repeatedly stressing that this does *not* require any new overloads of `any_of` to allow this usage. The above function evaluates exactly as:

```

auto dangerous_teams_rewritten(std::string const& s) -> bool {
    return ranges::any_of(
        ranges::transform(
            ranges::group_by(
                s,
                std::equal_to{ }),
            ranges::distance),
        [](std::size_t s){
            return s >= 7;
        });
}

```

This rewrite isn't exactly readable, but that's not the point - nobody has to read it. Only the compiler has to know how to evaluate these calls, and it has no problem at all figuring out the right thing to do.

§ 4.2.7 Async Examples

Ranges isn't the only subset of C++ that would benefit from the existence of a pipeline rewrite operator. At CppCon 2019, Eric Niebler and David Hollman presented [A Unifying Abstraction for Async in C++](#), illustrating the executors work that's been ongoing for a few years now. They build up to the following example:

```
int main() {
    auto f = async_algo(new_thread());
    auto f2 = then(f, [](int i){
        return i + rand();
    });
    printf("%d\n", sync_wait<int>(f2));
}
```

With this proposal, this could be written (with zero additional library work on anyone's part) as:

```
int main() {
    auto result =
        new_thread()
        |> async_algo()
        |> then([](int i){ return i + rand(); })
        |> sync_wait<int>();
    printf("%d\n", result);
}
```

Which demonstrates the linear flow of execution quite well.

Here's a more realistic example from libunifex [\[libunifex\]](#) (I changed the printf strings just to fit side-by-side better, in the original code they are more meaningful than A, B, C):

Existing Code	With Pipeline
<pre>auto start = std::chrono::steady_clock::now(); inplace_stop_source timerStopSource; sync_wait(with_query_value(when_all(transform(schedule_at(scheduler, now(scheduler) + 1s), []() { std::printf("A"); }, transform(schedule_at(scheduler, now(scheduler) + 2s), []() { std::printf("B"); }, transform(schedule_at(scheduler, now(scheduler) + 1500ms), [&]() { std::printf("C\n"); timerStopSource.request_stop(); }, get_stop_token,</pre>	<pre>auto start = std::chrono::steady_clock::now(); inplace_stop_source timerStopSource; when_all(scheduler > schedule_at(now(scheduler) + 1s) > transform([]() { std::printf("A\n"); }, scheduler > schedule_at(now(scheduler) + 2s) > transform([]() { std::printf("B\n"); }, scheduler > schedule_at(now(scheduler) + 1500ms) > transform([&]() { std::printf("C\n"); timerStopSource.request_stop(); }))) > with_query_value(get_stop_token,</pre>

Existing Code	With Pipeline
<pre>timerStopSource.get_token()); auto end = std::chrono::steady_clock::now();</pre>	<pre>timerStopSource.get_token()) > sync_wait(); auto end = std::chrono::steady_clock::now();</pre>

§ 4.3 Prior Art (in C++ and elsewhere)

It's important to point out that the notion of a pipeline rewrite operator is not novel across programming languages, and it isn't even novel in C++.

§ 4.3.1 Elixir

The particular form of the operator this paper is proposing comes from the Elixir programming language, where it is known as the pipe operator [[elixir.pipe](#)]. Its semantics are the same as are being proposed here. From the docs:

```
iex> "Elixir rocks" |> String.upcase() |> String.split()
["ELIXIR", "ROCKS"]
```

§ 4.3.2 Hack

Another language with the same operator is Hack, except its usage is slightly more generalized than either Elixir's or what is being proposed here [[hack.pipe](#)]. While the underlying idea is the same - a function call is split such that one argument is on the left of `|>` and the rest of the call is on the right - Hack instead stores the left-hand operand in a variable named `$$` which then must appear on the right-hand side (but doesn't necessarily have to be the first argument):

```
$x = vec[2,1,3]
|> Vec\map($$, $a ==> $a * $a)
|> Vec\sort($$);
```

§ 4.3.3 F#, Julia, OCaml, Elm

F# [[f-sharp.pipe](#)], Julia [[julia.pipe](#)], Elm [[elm.pipe](#)], and OCaml also have an operator named `|>` - but theirs is slightly different. Theirs all invoke the right-hand side with the left-hand side as its sole argument: rather than `x |> f(y)` meaning `f(x, y)` as is being proposed here and as it means in Elixir, it instead means `f(y)(x)`. In other words, given a binary operator as proposed in [[P1282R0](#)], these languages' version of `|>` could be implemented as a global:

```
template <typename Arg, std::invocable<Arg> F>
auto operator|>(Arg&& a, F&& f)
-> std::invoke_result_t<F, Arg>
{
    return std::invoke(FWD(f), FWD(a));
}
```

This may be more in line with how most operators in C++ work, but it's also not especially useful. It would make it marginally easier to implement partial calls - you could just return a lambda from those

partial calls instead of having to have this left-pipeable type - but it's still a lot of work on everyone's behalf to get there.

§ 4.3.4 JavaScript

JavaScript is currently discussion a proposal for an operator named `|>` [[javascript.pipeline](#)]. Indeed, they have two different directions for the proposal that they are considering (really three, but for our purposes two of them are basically equivalent):

1. What F# does: where `x |> f(y)` evaluates as `f(y)(x)`.
2. A very expanded version of what Hack does, where the left-hand argument needs a placeholder but right-hand side no longer needs to be a function. An example from the paper

Existing Code	With Pipeline Rewrite
<pre>console.log(await stream.write(new User.Message(capitalize(doubledSay(await promise throw new TypeError(`Invalid value from                 \${promise}`)), ', ') + '!')));</pre>	<pre>promise > await # > # throw new TypeError(`Invalid value from       \${promise}`) > doubleSay(#, ', ') > capitalize > # + '!' > new User.Message(#) > await stream.write(#) > console.log;</pre>

§ 4.3.5 Clojure

Clojure solves this problem in an entirely different way, that is still worth noting for completeness. It has threading operators spelled `->` [[clojure.thread-first](#)] and `->>` [[clojure.thread-last](#)]. The former operator sends every argument into the first parameter of the subsequent function call (quite like `|>` in Elixir and `|` in Ranges, except it only appears at the front of an expression) and the latter sends every argument in to the *last* parameter of the subsequent function call. Here are a few examples from those docs:

```
;; Arguably a bit cumbersome to read:
user=> (first (.split (.replace (.toUpperCase "a b c d") "A" "X") " "))
"X"

;; Perhaps easier to read:
user=> (-> "a b c d"
          .toUpperCase
          (.replace "A" "X")
          (.split " ")
          first)
"X"

;; An example of using the "thread-last" macro to get
```

```
;; the sum of the first 10 even squares.
user=> (->> (range)
          (map #(* % %))
          (filter even?)
          (take 10)
          (reduce +))

1140

;; This expands to:
user=> (reduce +
            (take 10
                  (filter even?
                        (map #(* % %)
                            (range))))))

1140
```

§ 4.3.6 C++

As far as C++ is concerned, it would be a third in a set of operators that have special semantics as far as function lookup is concerned:

```
x->f(y)
x.f(y)
x |> f(y)
```

None of the first two operators evaluate `f(y)` by itself and then evaluate something else joining that result with `x`. `x->f(y)` might invoke something named `operator->`, but once it finds a pointer, we do a single function call to something named `f` using both arguments. It's just that while the first two always invoke a member function, the last would always invoke a non-member function.

§ 5 Operator Precedence

An important question (discussed at some length on the reflectors [\[ext.precedence\]](#)) is where in the C++ grammar `|>` should go into. We'll start by copying the operator precedence table from [cppreference](#) [\[cppref.precedence\]](#) and adding into it where other languages' versions of `|>` appear (thereby providing precedence to the precedence question - note that the placements for other languages are our best approximation for how copying that language would fit into our grammar):

Precedence	Operator
1	<code>::</code>
2	<code>a++ a--</code> <code>T() T{}</code> <code>a() a[]</code> <code>. -></code> <code>- P2011R0 -</code>
3	<code>++a --a</code> <code>+a -a</code> <code>! ~</code> <code>(T)</code>

	<code>*a &a</code> <code>sizeof</code> <code>co_await</code> <code>new new[]</code> <code>delete delete[]</code>
4	<code>. * -> *</code>
5	<code>a * b a / b a % b</code>
6	<code>a + b a - b</code>
7	<code><< >></code> – Elixir, F#, OCaml –
8	<code><= ></code>
9	<code>< <=</code> <code>> >=</code>
10	<code>== !=</code>
11	<code>&</code>
12	<code>^</code>
13	<code> </code>
14	<code>&&</code>
15	<code> </code>
15.5	– JavaScript, Hack, Elm –
16	<code>a ? b : c</code> <code>throw</code> <code>co_yield</code> <code>=</code> <code>op=</code>
17	<code>,</code>

What we see are that, ignoring the first draft of this proposal, there are two places where languages decided to place this operator: just below the math operators, and basically as low as possible.

Precedence needs to be driven by usage, and what users might expect a given syntax to look like. Consider unary operator, this example courtesy of Richard Smith:

```

++x |> f()
-x  |> f()
-3  |> f()
*x  |> f()

```

The expectation is likely quite strong that these evaluate as `f(++x)`, `f(-x)`, `f(-3)`, and `f(*x)`, respectively, while in the first draft of the paper they would have evaluated as `++f(x)`, `-f(x)`, `-f(3)`, and `*f(x)`. This suggests that treating this as a new *postfix-expression* is simply the wrong model. Similarly, having `x |> f()` ++ evaluate as `f(x)++` seems surprising - since it looks very much like `f()++` is the function intended to be evaluated.

Further than that, the question becomes less obvious, but it seems like there are three reasonable levels for this operator to sit:

- At 4.5: Below `.*` and `->*` but above all the other binary operators.
- At 6.5: Where Elixir, F#, and OCaml have it. Below the math, but above the comparisons.
- At 15.5: Where JavaScript, Hack, and Elm have it. As low as possible, just above assignment.

Before we delve further into precedence question, let's consider the situation with the left- and right-hand sides of `|>`. Unlike all the other binary operators, we're not evaluating both sides separately and then combining them with some function. Here, the right-hand side has to be something that is shaped like a function call - that isn't evaluated until we first evaluate the left hand side and then treat it as an argument.

This is straightforward to reason about in all the typical `|>` examples since every right-hand side is just a call. In:

```
return s
  |> views::group_by(std::equal_to{})
  |> views::transform(ranges::distance)
  |> ranges::any_of([])(std::size_t s){
    return s >= 7;
  });
```

we have basically `s |> f(x) |> g(y) |> h(z)`, nothing especially complicated. But how do we deal with more complex examples?

We think the appropriate model for handling the right-hand side is that it must look like a call expression (that is, it must look like `f()`) when we parse up to the appropriate precedence, and then we insert the left-hand side as the first argument of that call expression. This notably differs from the first draft of this paper, where `|>` was presented as another *postfix-expression*.

Let's consider several more complex examples, some courtesy of Davis Herring and most courtesy of Arthur O'Dwyer [[odwyer.precedence](#)], and look at how they might evaluate with differing precedence (and including R0 as the last column for comparison). For most of the examples, the choice of precedence doesn't matter once it's below the postfix operators – since the examples themselves mostly use postfix or unary operators – so for convenience of presentation (and to avoid duplication), we're splitting the examples in two tables: the first table only uses postfix and unary prefix operators while the second table uses some other operators.

Note that R0 of this paper only allowed function calls to be pipelined into - while this paper expands into all postfix parenthesized expression. That choice has nothing to do with choice of precedence though, so for the sake of interest, this table is presented as if R0 made the same choice.

Example	This Paper (R1)	Postfix (R0)
<code>x > f()</code>	<code>f(x)</code>	<code>f(x)</code>
<code>x > f()()</code>	<code>f()(x)</code>	<code>f(x)()</code>
<code>x > f().g</code>	ill-formed	<code>f(x).g</code>
<code>x > f().g()</code>	<code>f().g(x)</code>	<code>f(x).g()</code>

Example	This Paper (R1)	Postfix (R0)
x > (f()).g()	f().g(x)	ill-formed
r > filter(f) > transform(g)	transform(filter(r,f),g)	transform(filter(r,f),g)
x > f() > g<0>(0)	g<0>(f(x), 0)	g<0>(f(x), 0)
x > T::m()	T::m(x)	T::m(x)
x > T{}.m()	T{}.m(x)	ill-formed
x > T().m()	T().m(x)	T(x).m()
x > c.f()	c.f(x)	c.f(x)
x > getA().*getMemptr()	getA().*getMemptr(x)	getA(x).*getMemptr()
x > always(y)(z)	always(y)(x, z)	always(x, y)(z)
x > always(y)() > split()	split(always(y)(x))	split(always(x, y)())
x > get()++	ill-formed	get(x)++
x > ++get()	ill-formed	ill-formed
++x > get()	get(++x)	++get(x)
x > (y > z())	ill-formed	ill-formed
x > f().g<0>(0)	f().g<0>(x, 0)	f(x).g<0>(0)
-3 > std::abs()	std::abs(-3)	-std::abs(3)
co_await x > via(e)	via(co_await x, e)	co_await via(x, e)
co_yield x > via(e)	co_yield via(x, e)	co_yield via(x, e)
throw x > via(e)	throw via(x, e)	throw via(x, e)
return x > via(e)	return via(x, e)	return via(x, e)
s > rev() > find_if(a).base()	find_if(a).base(rev(s))	find_if(rev(s), a).base()
x > (f())	ill-formed	ill-formed
x > get()[i]	ill-formed	get(x)[i]
x > v[i]()	v[i](x)	ill-formed
x > v[i]()()	v[i]()()	ill-formed
(x > v[i]()())	v[i](x)()	ill-formed
x > (v[i]()())	v[i]()()	v[i](x)()
x > y.operator+()	y.operator+(x)	ill-formed
x > +y	ill-formed	ill-formed
c ? left : right > split('/')	c ? left : split(right, '/')	c ? left : split(right, '/')
(c ? left : right) > split('/')	split(c ? left : right)	split(c ? left : right)
c ? left > split('/') : right	c ? split('/', left) : right	c ? split('/', left) : right
x > f() > std::make_pair(y)	std::make_pair(f(x), y)	std::make_pair(f(x), y)

Example	This Paper (R1)	Postfix (R0)
x > f() > std::pair<X,Y>(y)	std::pair<X,Y>(x, y)	std::pair<X,Y>(x, y)
x > new T()	ill-formed	ill-formed
x > [](int x, int y){ return x+y; }(1)	[](int x, int y){ return x+y; }(x,1)	[](int x, int y){ return x+y; }(x,1)
x > f() > std::plus{} (1)	std::plus{}(f(x),1)	std::plus{}(f(x),1)

And a table illustrating different precedences:

Example	Above + (4.5)	Between + and == (6.5)	Below == (15.5)	Postfix (R0) (2)
x + y > f()	x + f(y)	f(x + y)	f(x + y)	x + f(y)
(x + y) > f()	f(x + y)	f(x + y)	f(x + y)	f(x + y)
ctr > size() == max()	size(ctr) == max()	size(ctr) == max()	ill-formed	size(ctr) == max()
(ctr > size()) == max()	size(ctr) == max()	size(ctr) == max()	size(ctr) == max()	size(ctr) == max()
x > f() + g()	f(x) + g()	ill-formed	ill-formed	f(x) + g()
x > f() + 3	f(x) + 3	ill-formed	ill-formed	f(x) + 3
(x > f()) + 3	f(x) + 3	f(x) + 3	f(x) + 3	f(x) + 3
"hi"sv > count('o') == 0	count("hi"sv, 'o') == 0	count("hi"sv, 'o') == 0	ill-formed	count("hi"sv, 'o') == 0
("hi"sv > count('o')) == 0	count("hi"sv, 'o') == 0	count("hi"sv, 'o') == 0	count("hi"sv, 'o') == 0	count("hi"sv, 'o') == 0
v > filter(2) > size() == 1	size(filter(v, 2)) == 0	size(filter(v, 2)) == 0	ill-formed	size(filter(v, 2)) == 0
(v > filter(2) > size()) == 1	size(filter(v, 2)) == 0	size(filter(v, 2)) == 0	size(filter(v, 2)) == 0	size(filter(v, 2)) == 0

Example	Above + (4.5)	Between + and == (6.5)	Below == (15.5)	Postfix (R0) (2)
a > b() - c > d()	b(a) - d(c)	ill-formed	ill-formed	b(a) - d(c)
a > b() c() > d()	b(a) d(c())	b(a) d(c())	ill-formed	b(a) d(c())
x + y > f() + g() > a.h()	x + f(y) + a.h(g())	ill-formed	ill-formed	ill-formed

Consider `x |> f() + g()`. If `|>` has precedence above `+`, then the right-hand side would be `f()`. That's a call expression, which makes this valid. But if `|>` has lower precedence, then the right hand side is `f() + g()` - which is *not* a call expression (it's an addition). That's ill-formed.

The same analysis holds for `ctr |> size() == max()`, just with a different operator.

Let us draw your attention to two of the examples above:

- `x |> f() + y` is described as being either `f(x) + y` or ill-formed
- `x + y |> f()` is described as being either `x + f(y)` or `f(x + y)`

Is it not possible to have `f(x) + y` for the first example and `f(x + y)` for the second? In other words, is it possible to have different precedence on each side of `|>` (in this case, lower than `+` on the left but higher than `+` on the right)? We think that would just be very confusing, not to mention difficult to specify. It's already hard to keep track of operator precedence, but this would bring in an entirely novel problem which is that in `x + y |> f() + z()`, this would then evaluate as `f(x + y) + z()` and you would have the two `+`s differ in their precedence to the `|>`? We're not sure what the mental model for that would be.

For the paper itself, we propose the precedence of `|>` to be at 4.5: just below `.*` and `->*`. But a significant argument in favor of lower precedence comes from considering a different direction for this operator... an explicit placeholder.

§ 6 A Placeholder syntax

This proposal (along with languages like Elixir) is for `x |> f(y)` to evaluate as `f(x, y)`: the left-hand argument always gets put in the first slot of the call expression on the right-hand side. This feature has a lot of utility in a wide variety of contexts.

But what if you want to put the left-hand argument somewhere else? While `|>` can allow pipelining into `views::zip`, it would not be able to allow pipelining into `views::zip_with` - there the first parameter is a transform operator. We would write `views::zip_with(plus{}, a, b)`, and `plus{} |> views::zip_with(a, b)` is unlikely to ever actually be written.

This is where a placeholder syntax would come in handy. If, instead of requiring a function call (that the left-hand side was inserted into), we chose to require a placeholder (like Hack and one of the JavaScript

proposals), we could have both:

```
a |> views::zip(>, b)           // evaluates as views::zip(a, b)
a |> views::zip_with(plus{}, >, b) // evaluates as views::zip_with(plus{}, a, b)
```

Here, we’re using `>` as a placeholder, as suggested by Daveed Vandevoorde (this choice of placeholder is itself simply a placeholder. Other placeholders we considered were `%` and `^`, which unfortunately can run into ambiguities with C++/CLI. `#` is unlikely to be confused for a preprocessor directive? `%%`?)

Moreover, as JavaScript demonstrates for us already, with the placeholder approach, we wouldn’t actually need to keep the requirement that the right-hand side is a call expression. It could be any expression at all, as long as it contains precisely one `>`. `a |> 2 * >` could be a valid expression that means exactly `2 * a`.

The main benefit of a direction pursuing placeholder syntax is that the syntax can be used with any function, and indeed with any expression. As with `zip_with`, you don’t need to rely on the function you intend on calling having the the correct first parameter. This would allow the `FILE` example described [later](#) to be written entirely `FILE`-first:

```
FILE* file = fopen( "a.txt", "wb" );
if (file) {
    file |> fputs("Hello world", >);
    file |> fseek(>, 9, SEEK_SET);
    file |> fclose(>);
}
```

An the earlier `trim` example could change to be fully pipelined (that is, we can take the extra `.base()` at the end without having to introduce parentheses - it just goes at the end of the expression, since that’s the last thing logically we need to do):

No placeholder	Placeholder
<pre>auto trim(std::string const& str) -> std::string { auto b = str > ranges::find_if(isalpha); auto e = str > views::reverse() > ranges::find_if(isalpha); return std::string(b, e.base()); }</pre>	<pre>auto trim(std::string const& str) -> std::string { auto b = str > ranges::find_if(>, isalpha); auto e = str > views::reverse(>) > ranges::find_if(>, isalpha) > >.base(); return std::string(b, e); }</pre>

The major cost of this direction is that we add more syntax to what would be by far the most common use case: the `x |> f(y)` as `f(x, y)` examples used throughout this proposal. In the above `trim` example, three of the four `>`s are used as the first parameter, for instance.

Unless we could optimize for this case, and come up with a way to allow both syntaxes (as below), we’re hesitant to be fully on board.

```
x |> f(y)           // f(x, y)
x |> f(>, y)        // f(x, y)
```



```
x |> f(y, >) // f(y, x)
```

But it's quite important that we consider this now, since this would inform the choice of precedence. If we ever want to go in the direction of placeholder syntax, it's quite valuable for the precedence of `|>` to be as low as possible. With placeholders, such a choice of precedence allows `|>` to behave as an operator separator - and allows you to write whatever flow of operations you want to write without thoughts to precedence at all. Look again at the [JavaScript example](#) presented earlier. There's a lot of different operations going on in that example - but the combined use of `|>` and placeholder allows for a direct, linear flow... top down.

§ 7 Concerns with the Pipeline Operator

There are two major concerns regarding a pipeline operator that need to be discussed:

1. Why can't we just have unified function call syntax?
2. Ranges already has a pipeline operator, that additionally supports composition. What about composition, and what should we do about Ranges going forward?

§ 7.1 What about Unified Function Call Syntax?

Any discussion regarding pipeline rewrites, or even the pipeline syntax in C++20 ranges, will eventually hit on Unified Function Call Syntax (UFCS). As previously described in a blog post [\[revzin\]](#), UFCS means different things to different people: there were quite a few proposals under this banner that had a variety of different semantics and different applicability to this problem space.

In [\[N4165\]](#), Herb Sutter presented two goals:

Enable more-generic code: Today, generic code cannot invoke a function on a `T` object without knowing whether the function is a member or non-member, and must commit to one. This is long-standing known issue in C++.

and

Enable “extension methods” without a separate one-off language feature: The proposed generalization enables calling non-member functions (and function pointers, function objects, etc.) symmetrically with member functions, but without a separate and more limited “extension methods” language feature. Further, unlike “extension methods” in other languages which are a special-purpose feature that adds only the ability to add member functions to an existing class, this proposal would immediately work with calling existing library code without any change. (See also following points.)

We will address these two points in turn.

§ 7.1.1 UFCS does not enable more-generic code

The most consistent argument in favor of UFCS, regardless of proposal details, as Herb made here and Bjarne Stroustrup made in [N4174] and they both made together in [N4474] and Bjarne made again in a blog post on isocpp.org [stroustrup] is this one, quoting from Bjarne's blog post:

C++ provides two calling syntaxes, `x.f(y)` and `f(x,y)`. This has bothered me for a long time. I discussed the problem in the context of multimethods in D&E in 1994, referring back to a proposal by Doug Lea from 1991, and again in 2007. Each of the syntaxes has its virtues, but they force design decisions on people, especially library designers. When you write a library, which syntax do you use for operations on objects passed to you by your users? The STL insists on the traditional functional syntax, `f(x,y)`, whereas many OO libraries insist on the dot notation, `x.f(y)`. In turn, libraries force these decisions on their users.

This is a very real problem in writing generic code in C++, one which UFCS set out to solve. Since a lot of the examples in this paper deal with Ranges already, let's stick to Ranges.

§ 7.1.1.1 Fails for fundamental types

The fundamental concept in C++20 Ranges is the `std::range` concept. With a version of UFCS in which member function call syntax could find non-member functions (notably, not the version that was voted on in plenary - that one allowed non-member call syntax to find member functions only), we might expect to be able to define that concept this way:

```
template <typename R>
concept ufcs_range = requires (R& rng) {
    { rng.begin() } -> input_or_output_iterator;
    { rng.end() } -> sentinel_for<decltype(rng.begin())>;
}
```

You don't even need any language changes for this to work with all the standard library containers - those just have member `begin` and `end` directly. `ufcs_range<vector<int>>` is satisfied without much fuss.

For a type that looks like:

```
namespace lib {
    struct some_container { /* ... */ };
    struct some_iterator { /* ... */ };

    auto begin(some_container&) -> some_iterator;
    auto end(some_container&) -> some_iterator;
}
```

`ufcs_range<lib::some_container>` would be satisfied by saying that lookup for `rng.begin()` would find the free function `lib::begin()` (let's assume for simplicity that `some_container` has no members named `begin` or `end`) and likewise for `rng.end()`.

This seems to work, what's the problem?

Consider `ufcs_range<lib::some_container[10]>`. Is this satisfied? C arrays have no member functions, so the member lookup trivially fails. But the only candidate we find with ADL for `begin` isn't viable - we don't have a candidate that can take a C array. Consider what we have to do in order to make this work.

The way we make `begin()` and `end()` work for C arrays is completely agnostic to the type that the array is of. It shouldn't be `lib`'s responsibility to provide this function.

And even if it did, consider `ufcs_range<int[10]>`. Here, not only do we have no member functions but we also have no associated namespaces in which to look for what we need! Do we conclude that `int[10]` is not a range? The only way for `rng.begin()` to work on a simple array of `ints` with UFCS is to have `begin` in scope - which means either that it has to be a global function with no intervening declarations of `begin` (not going to happen) or it's up to every algorithm to bring them into scope. Something like:

```
namespace std {
    template <typename T, size_t N>
    auto begin(T (&arr)[N]) -> T* { return arr; }
    template <typename T, size_t N>
    auto end(T (&arr)[N]) -> T* { return arr+N; }

    template <typename R>
    concept ufcs_range = requires (R& rng) {
        { rng.begin() } -> input_or_output_iterator;
        { rng.end() } -> sentinel_for<decltype(rng.begin())>;
    }
}

template <std::range R, typename Value>
auto find(R& r, Value const& value)
{
    using std::begin, std::end;
    // UFCS logic here, regular lookup will find the
    // begin/end brought in with the using-declarations
    auto first = r.begin();
    auto last = r.end();
    // ...
}
```

But if we have to do this dance *anyway*, we didn't gain anything from UFCS at all. We can do the exact same thing already, at the cost of a few more lines of code (adding declarations of `begin` and `end` that invoke member functions) and using non-member syntax instead.

And if we don't have a generic solution that works for the fundamental types (this example fails for raw arrays, but the general solution will fail if you want to provide implementations for a generic algorithm for types like `int` or `char` or pointers), then we don't have a generic solution.

§ 7.1.1.2 Too greedy on names

UFCS works by having the non-member fallback find a free function of the same name with ADL. C++20 ranges introduces customization point objects that do this for you, which at least makes it easier to write new algorithms (although the customization point objects themselves are a chore to implement). But this has fundamental problems, as noted in [P1895R0]:

1. Each one internally dispatches via ADL to a free function of the same name, which has the effect of globally reserving that identifier (within some constraints). Two independent libraries that pick the same name for an ADL customization point still risk collision.

2. There is occasionally a need to write wrapper types that ought to be transparent to customization. (Type-erasing wrappers are one such example.) With C++20's CPOs, there is no way to generically forward customizations through the transparent wrappers

Point (1) above is an immediate and pressing concern in the executors design, where we would like to give platform authors the ability to directly customize parallel algorithms. Using the C++20 CPO design would explode the number of uniquely-named ADL customization points from a handful to potentially hundreds, which would create havoc for the ecosystem.

Point (2) is also a concern for executors, where platform authors would like to decorate executor types with platform-specific "properties" (extra-standard affinities and thresholds of all sorts) that can be exposed even through transparent layers of adaptation, such as the polymorphic executor wrapper. This need led to the properties system (P1393) which LEWG has already reviewed. It's important to note that, although the problems in C++20 CPOs are exposed by the executors work, the problems are not specific to executors.

The paper expresses its concerns specifically in relation to CPOs, but the problem is really about using ADL for customization points. UFCS as a language feature would push much harder in that direction, and it's not a good direction.

Put differently, designing generic code on top of ADL is somewhat akin to just eschewing namespaces altogether and going back to our C roots.

§ 7.1.1.3 This is a concepts problem

As argued in [P1900R0], the problem of customization should be considered a [concepts](#) problem and merits a [concepts](#) solution. What we are trying to do with this paper has nothing to do with customization - we are not trying to solve, or even address, this problem.

Instead, what we are trying to do is address...

§ 7.1.2 UFCS does enable extension methods without a separate one-off language feature

The other argument that Herb made in favor of UFCS was in favor of adopting extension method without a special language feature specific to them. Notably, the argument he made in [N4165] specifically cites the desire to avoid having to make any library changes to start using all the functionality - which is precisely the argument we are making with this proposal with regards to pipelines.

Herb also makes the argument that allow member call syntax to lookup non-member functions is friendlier to developers and tools as it allows things like autocomplete to work.

But the question we have is: why does this feature *need* to be spelled `.`? There are many problems with that specific choice that completely evaporate if we simply choose an alternate spelling. There is no question about how the candidate set should be considered, or how overload resolution should work, nor is there any concern about long term library work and accidentally breaking user code by adding a private member function. The alternate spelling this paper is proposing is `|>`.

One example from that document is:

```
FILE* file = fopen( "a.txt", "wb" );
if (file) {
    fputs("Hello world", file);
    fseek(file, 9, SEEK_SET);
    fclose(file);
}
```

This proposal wouldn't allow for any help on `fputs`. To do that, we would need the [placeholder syntax](#) described earlier to allow for `file |> fputs("Hello world", >)`. But the rest could be written as:

```
FILE* file = fopen( "a.txt", "wb" );
if (file) {
    fputs("Hello world", file);
    file |> fseek(9, SEEK_SET);
    file |> fclose();
}
```

§ 7.1.3 1 Syntax, 2 Meanings

Fundamentally, a problem with UFCS is that it would be yet another example in C++ where a single syntax has two potential meanings: `x.f()` would either be a member function call or a non-member function call, it simply depends.

We already have several examples of this kind of thing in the language, and they are very frequent sources of complaint by beginners and experts alike:

- `T&&` is either an rvalue reference or a forwarding reference, depending on where `T` comes from.
- `x{v}` can construct an `initializer_list` from `v`, or not, depending on `x`.

Name lookup and overload resolution are two of the most complex aspects of a very complex language. Instead of pursuing UFCS, we are proposing a second syntax that actually has the same meaning as an already existing syntax: `x |> f()` has the exact same meaning as `f(x)`. But both of these syntaxes always mean invoking the free function `f` and `x.f()` always means invoking the member function `f`, and having that differentiation seems like a positive thing rather than a negative thing.

§ 7.2 What about pipeline composition?

This paper largely focuses on the idea from Ranges that you can have a total function call and a partial function call:

```
// total call
views::filter(ints, is_even);

// partial call
ints | views::filter(is_even);
```

But there's another important aspect of the Ranges design that it's important to discuss: you can build pipelines even without the function call part. That is, we can just write:

```
auto even = views::filter(is_even);
```

We don't need to "immediately invoke" this filter on a range, we can hold onto it. We can have a function that returns that filter. And we can build a pipeline from a large number of range adapters without even having an input range:

```
// In: range<Session>
// Out: range<int> for the logged in sessions
auto session_traffic()
{
    return views::filter(&Session::is_logged_on)
        | views::transform(&Session::num_messages);
}

auto traffic = accumulate(my_sessions | session_traffic());
```

Note the comments: it says that the input is a `range<Session>`. Where is it? It's nowhere.

What's actually going on here is, effectively, a big foray by C++ into the world of partial function composition. Now we have become Haskell. This is a very cool, very useful, bit of functionality that the range adapters provide.

The pipeline rewrite operator we are proposing does not do this. You would have to write the simple even-filter as a lambda:

```
auto even = [](auto&& rng) { return rng |> views::filter(is_even); };
```

And you would have to write the `session_traffic` example this way:

```
// In: range<Session>
// Out: range<int> for the logged in sessions
template <typename Range>
auto session_traffic(Range&& rng)
{
    return FWD(rng)
        |> views::filter(&Session::is_logged_on)
        |> views::transform(&Session::num_messages);
}
```

Notably, the formatter becomes a function template since now we actually need to express the input argument directly. This actually allows us to add a constraint on `session_traffic` that the parameter `rng` is actually a `range<Session>`, as the comment indicates it must be:

```
template <std::range R>
    requires std::is_same_v<std::range_value_t<R>, Session>
auto session_traffic(R&&);
```

On the other hand, this is more that we have to write every time.

This would certainly open up questions about how we want to handle range adapters in the future if we choose to adopt this proposal. The above code cannot work without the input range (`views::filter(f) |> views::transform(g)` would not work). The only way to preserve the composition of range adapters as separate from the range input would be to preserve the current implementation of `operator|`.

However, the utility of partial function application and composition is much, much more expansive than just range adapters. And if we think it's a valuable thing for range adapters, maybe we should find a way to make it work for all other C++ applications?

§ 7.3 What to do about Ranges going forward?

An important question this paper needs to answer is: let's say we adopt `|>` as proposed. With the notable exception of the adapter compositions described in the previous section, `|>` would completely subsume the use of `|` and we would want to encourage its use going forward (especially since `|>` would not be intersperse-able with `|`, you'd have to switch to `|>` if you want to continue your pipeline into the algorithms).

So what, then, do we do with Ranges in C++23? We fully expect many more view adapters to be added into the standard library in this time frame - should those view adapters support `|`?

We see three alternatives.

1. Ranges could complete ignore `|>`. All new view adapters should add `|` anyway.
2. Don't add `|` support to any new C++23 views, keep `|` for the existing ones.
3. Don't add `|` support to any new C++23 views, and deprecate `|` for the existing ones.

The advantage of deprecation is that we really would only want one way to do something - and `|>` is a superior pipeline tool to `|`.

But deprecation has cost. Even though we're still in C++20, standard libraries will ship Ranges implementations this year, and this proposal could not be adopted as part of C++23 until February 2021 at the earliest - and code will certainly be written that uses Ranges with pipelines. Even with `|>`, that code will continue to be perfectly functional and correct. If we deprecate `|`, users may be in a position where they have code that has to compile against one compiler that supports `|>` and one that doesn't. Deprecation warnings seem like they would make for an unnecessarily difficult situation for early adopters (unless we very nicely suggest and encourage all the implementations to provide a flag to specifically disabling the deprecation of this specific feature - otherwise users might have to just disable all deprecation warnings, which seems inherently undesirable).

We think, and Eric Niebler in private correspondence agrees, that the right option here is (1): Ranges should continue adding `|` for new view adapters for consistency and not deprecate anything. This may seem at odds with one of the benefits of `|>` that we laid out earlier - that the existence of the library machinery adds overhead to compiler throughput.

Because it kind of is.

But we hope to investigate, in range-v3 to start, the option of having a macro opt-out of defining the `|` support entirely. That is, rather than deprecate `|` and force users to move forward - possibly running into the kinds of problems mentioned earlier - let users move forward at their own pace and disable the costs when they don't need them anymore. This seems like a much softer way to move forward.

§ 7.4 Other Concerns

Some C++20 code could break. In the same way that the introduction of `operator<=>` introduced a `<=>` token that would break code that passed the address of an `operator<=` as a template argument (`f<&operator<=>()` would now have to be written as `f<&operator<= >()`), a `|>` token would break code that passes the address of an `operator|` as a template argument (`g<&operator|>()` would now have to be written as `g<&operator| >()`). This wasn't a huge concern with the spaceship operator, and it isn't a huge concern with the pipeline rewrite operator.

There may be a concern that this would lead to yet another conflict in the C++ community as to whether the proper way to invoke functions is spelled `west(invocable)` or `invocable |> east()`. We're not too concerned about this potential conflict. Just wanted to be thorough.

§ 8 Wording

This wording is based on adding `|>` with precedence just below `.*` and `->*`.

Add `|>` as a token to 5.12 [\[lex.operators\]](#).

Change the grammar in *multiplicative-expression* to refer to a new production *pipeline-expression* instead 7.6.5 [\[expr.mul\]](#):

```

multiplicative-expression:
- pm-expression
- multiplicative-expression * pm-expression
- multiplicative-expression / pm-expression
- multiplicative-expression % pm-expression
+ pipeline-expression
+ multiplicative-expression * pipeline-expression
+ multiplicative-expression / pipeline-expression
+ multiplicative-expression % pipeline-expression

```

Add a new section named “Pipeline rewrite” [\[expr.pizza\]](#):

```

pipeline-expression:
  pm-expression
  pipeline-expression |> pipeline-rhs

pipeline-rhs:
  pm-expression ( expression-listopt )
  simple-type-specifier ( expression-listopt )
  typename-specifier ( expression-listopt )
  dynamic_cast < type-id > ( )
  static_cast < type-id > ( )
  reinterpret_cast < type-id > ( )
  const_cast < type-id > ( )
  typeid ( )

```

- 1 An expression of the form `E1 |> E2(Eargs)`, where `E2` is a *pm-expression* and `Eargs` is a possibly empty, comma-separated list of *initializer-clauses*, is identical (by definition) to `E2(E1, Eargs)` ([\[expr.call\]](#)), except that `E1` is sequenced before `E2`. *[Note: E2 is still sequenced before the rest of the function arguments. -end note]*

- 2 An expression of the form `E1 |> simple-type-specifier(E_args)` is identical (by definition) to `simple-type-specifier(E1, E_args)` ([[expr.type.conv](#)]) except that `E1` is sequenced before `E_args`.
- 3 An expression of the form `E1 |> typename-specifier(E_args)` is identical (by definition) to `typename-specifier(E1, E_args)` ([[expr.type.conv](#)]) except that `E1` is sequenced before `E_args`.
- 4 An expression of the form `E |> dynamic_cast<typeid>()` is identical (by definition) to `dynamic_cast<typeid>(E)`.
- 5 An expression of the form `E |> static_cast<typeid>()` is identical (by definition) to `static_cast<typeid>(E)`.
- 6 An expression of the form `E |> reinterpret_cast<typeid>()` is identical (by definition) to `reinterpret_cast<typeid>(E)`.
- 7 An expression of the form `E |> typeid()` is identical (by definition) to `typeid(E)`.

Add `|>` to the list of non-overloadable operators in 12.6 [[over.oper](#)]/3:

- 3 The following operators cannot be overloaded:

`..* :: ?: |>`

nor can the preprocessing symbols `#` ([[cpp.stringize](#)]) and `##` ([[cpp.concat](#)]).

Add a new Annex C entry, mirroring the one that exists for `<=>` in C.1.1 [[diff.cpp17.lex](#)]/3:

Affected subclause: 5.12 [[lex.operators](#)] **Change:** New operator `|>`.

Rationale: Necessary for new functionality.

Effect on original feature: Valid C++ 2020 code that contains a `|` token immediately followed by a `>` token may be ill-formed or have different semantics in this International Standard:

```
namespace N {
    struct X {};
    bool operator|(X, X);
    template<bool(X, X)> struct Y {};
    Y<operator|> y;           // ill-formed; previously well-formed
}
```

§ 9 Acknowledgments

Several people helped enormously with shaping this paper, both with direct feedback and giving us more information about other languages: Gašper Ažman, Davis Herring, Arthur O'Dwyer, Tim Song, Richard Smith, Faisal Vali, Tony van Eerd, Daveed Vandevoorde, and Ville Voutilainen.

§ 10 References

[clojure.thread-first] ClojureDocs. 2010. -> clojure.core.

<https://clojuredocs.org/clojure.core/-/%3E>

[clojure.thread-last] ClojureDocs. 2010. ->> clojure.core.

<https://clojuredocs.org/clojure.core/-/%3E%3E>

[cppref.precedence] cppreference. 2020. C++ Operator Precedence.

https://en.cppreference.com/w/cpp/language/operator_precedence

[elixir.pipe] Elixir School. 2019. Pipe Operator - Elixir School.

<https://elixirschool.com/en/lessons/basics/pipe-operator/>

[elm.pipe] elm-lang. 2012. Basics.

[https://package.elm-lang.org/packages/elm/core/latest/Basics#\(|%3E\)](https://package.elm-lang.org/packages/elm/core/latest/Basics#(|%3E))

[ext.precedence] Ext Reflectors. 2020. Precedence for |> (P2011: pipeline rewrite operator).

<https://lists.isocpp.org/ext/2020/04/13071.php>

[f-sharp.pipe] F#. 2020. Function.

<https://docs.microsoft.com/en-us/dotnet/fsharp/language-reference/functions/>

[hack.pipe] HHVM. 2019. Expressions and Operators - Pipe.

<https://docs.hhvm.com/hack/expressions-and-operators/pipe>

[hoekstra] Conor Hoekstra. 2019. CppNow 2019: Algorithm Intuition.

<https://www.youtube.com/watch?v=48gV1SNm3WA>

[javascript.pipeline] TC39. 2019. Proposals for |> operator.

<https://github.com/tc39/proposal-pipeline-operator/wiki>

[julia.pipe] julialang. 2020. Function Composition and Piping.

<https://docs.julialang.org/en/v1/manual/functions/#Function-composition-and-piping-1>

[libunifex] libunifex. 2020. io_epoll_test.cpp, lines 89-108.

https://github.com/facebookexperimental/libunifex/blob/epoll-moar-sfinae/examples/linux/io_epoll_test.cpp#L89-L108

[N4165] Herb Sutter. 2014. Unified Call Syntax.

<https://wg21.link/n4165>

[N4174] Bjarne Stroustrup. 2014. Call syntax: x.f(y) vs. f(x,y).

<https://wg21.link/n4174>

[N4474] Bjarne Stroustrup, Herb Sutter. 2015. Unified Call Syntax: x.f(y) and f(x,y).

<https://wg21.link/n4474>

- [odwyer.precedence] Arthur O'Dwyer. 2020. Precedence of a proposed `|>` operator.
<https://quuxplusone.github.io/blog/2020/04/10/pipeline-operator-examples/>
- [P1282R0] Isabella Muerte. 2018. Ceci N'est Pas Une Pipe: Adding a workflow operator to C++.
<https://wg21.link/p1282r0>
- [P1895R0] Lewis Baker, Eric Niebler, Kirk Shoop. 2019. `tag_invoke`: A general pattern for supporting customisable functions.
<https://wg21.link/p1895r0>
- [P1900R0] Barry Revzin. 2019. Concepts-Adjacent Problems.
<https://wg21.link/p1900r0>
- [P2011R0] Barry Revzin, Colby Pike. 2020. A pipeline-rewrite operator.
<https://wg21.link/p2011r0>
- [prague.minutes] EWGI. 2020. Discussion of P2011R0 in Prague.
<http://wiki.edg.com/bin/view/Wg21prague/P2011R0SG17>
- [range.calendar] Eric Niebler. 2015. range-v3 calendar example.
<https://github.com/ericniebler/range-v3/blob/9221d364a82450873d49d302d475ce22110f0a9d/example/calendar.cpp>
- [range-v3] Eric Niebler. 2013. Range library for C++14/17/20, basis for C++20's `std::ranges`.
<https://github.com/ericniebler/range-v3/>
- [revzin] Barry Revzin. 2019. What is unified function call syntax anyway?
<https://brevzin.github.io/c++/2019/04/13/ufcs-history/>
- [stroustrup] Bjarne Stroustrup. 2016. A bit of background for the unified call proposal.
<https://isocpp.org/blog/2016/02/a-bit-of-background-for-the-unified-call-proposal>